SISO-UNAL: Gestión de cupos por prioridad en la inscripción de asignaturas de la Universidad Nacional de Colombia

Proyecto semestral de Estructuras de Datos — Entrega 1: propuesta, diseño e implementación inicial

Bryan Gustavo Llontop Angulo, Adrien Roho, Diego Fernando Castillo Albino
Facultad de Ingeniería, Universidad Nacional de Colombia
Sede Bogotá
Bogotá D.C., Colombia
{bllontop, aroho, dcastilloa}@unal.edu.co
Estructuras de Datos (DS3, grupo 03) — team-07 — 2026-2

***Resumen*—En la sede Bogotá de la Universidad Nacional de Colombia, cerca de 28.700 estudiantes de pregrado inscriben asignaturas mediante citas ordenadas por prioridad institucional. Después de la cita, esa prioridad se pierde: cuando un grupo está lleno no hay lista de espera, y cada cupo que se libera en el periodo de adiciones lo obtiene quien recargue primero el sistema. El objetivo del proyecto es diseñar e implementar en Java un sistema, SISO-UNAL, que complemente las citas con listas de espera por prioridad y reasignación automática de los cupos liberados, y que más adelante sugiera grupos alternativos y genere reportes de demanda insatisfecha. La solución usa estructuras de datos propias: una cola de prioridad sobre lista enlazada ordenada para la lista de espera, una lista enlazada para los inscritos, una cola FIFO para la cita y una pila para el carrito con deshacer. Cada estructura se usa a través de una interfaz para reemplazarla en las siguientes entregas por montículos, árboles, tablas hash y grafos. En esta primera entrega están implementadas la clasificación por prioridad, la inscripción con control de cupo, la lista de espera con aceptación previa del estudiante y la reasignación automática al cancelar.**

***Index Terms*—estructuras de datos, colas de prioridad, listas de espera, asignación de cupos, inscripción académica**

I. Introducción y planteamiento del problema

I-A. Contexto

La sede Bogotá de la Universidad Nacional de Colombia (UNAL) tiene 32.090 estudiantes matriculados, de los cuales 28.700 son de pregrado [1]. La Facultad de Ingeniería reúne cerca del 29 % (unos 8.320 estudiantes). Cada estudiante inscribe entre cuatro y cinco asignaturas, es decir, entre 115.000 y 143.500 inscripciones por semestre.

La inscripción se hace en el Sistema de Información Académica (SIA) y tiene cinco fases [3]: (1) consulta de la citación, que asigna a cada estudiante un día y una hora; (2) primera cita de inscripción; (3) adiciones y cancelaciones libres, en las semanas 1 y 2; (4) cancelación ordinaria, de la semana 3 a la 8, con pérdida de créditos y sin posibilidad de inscribir; y (5) cancelación extemporánea, por solicitud ante la facultad. El orden de las citas sigue cuatro grupos de prelación [2]: (1) discapacidad; (2) vulnerabilidad académica, por ejemplo PAPA $\leq$ 3,1; (3) avance curricular de al menos el 80 %; y (4) los demás, ordenados por el Índice de Prioridad de Asignación (IPA).

I-B. Situación que se desea resolver

Las citas deciden *cuándo* entra cada estudiante, pero la prioridad deja de tener efecto después:

- Cuando un grupo está lleno no hay lista de espera. En las adiciones (fase 3), cada cupo liberado lo obtiene quien recargue el SIA en ese momento, sin importar su prioridad.
- Si el grupo deseado está lleno, el estudiante debe buscar a mano otro grupo de la misma asignatura que no le cruce el horario.
- La universidad no sabe cuántos estudiantes quedaron sin una asignatura ni qué grupos tuvieron más demanda.

El problema es mayor en las asignaturas que comparten varias facultades (cálculo, inglés, libre elección) y en Ingeniería, por su tamaño.

Existe además el *sobrecupo*: cuando un grupo está en cero, el estudiante puede pedir un cupo adicional, que el docente o el Comité Asesor de Carrera aprueba caso por caso según el salón y la necesidad del estudiante. El sobrecupo es un mecanismo complementario que no afecta directamente este proyecto: crea cupos por excepción mediante un trámite manual, mientras que SISO-UNAL reasigna los cupos regulares que ya existen y se liberan.

I-C. Relevancia

El problema afecta a todo el pregrado cada semestre. Asignar cupos según la velocidad de conexión contradice la equidad que la universidad aplica en las citas y obliga a los estudiantes a vigilar el SIA durante días. Además, sin un registro de la demanda insatisfecha es difícil planear cuántos grupos abrir.

I-D. Dificultades computacionales

- **Escala:** decenas de miles de estudiantes y más de cien mil inscripciones en pocos días.
- **Orden por prioridad:** los estudiantes llegan a la espera en cualquier orden, pero deben salir según su prioridad, lo que exige una cola de prioridad eficiente.
- **Horarios:** un cupo solo puede asignarse a quien no tenga cruce de horario.
- **Consistencia:** un estudiante puede esperar en varios grupos de la misma asignatura y, al recibir cupo en uno, debe salir de los demás.

I-E. Propuesta general

SISO-UNAL no reemplaza las citas: las complementa con tres componentes.

1. **Lista de espera por prioridad:** cada grupo lleno tiene una espera ordenada por la prioridad institucional, y cada cupo liberado se asigna automáticamente al primero (Entrega 1).
2. **Grupos alternativos:** si un grupo está lleno, el sistema sugiere otro grupo de la misma asignatura con cupo y sin cruce (Entrega 2).
3. **Reportes de demanda:** los grupos y asignaturas con más estudiantes en espera, para planear cupos (Entregas 2 y 3).

II. Objetivos

II-A. Objetivo general

Diseñar e implementar en Java un sistema que complemente las citas de inscripción de la UNAL con listas de espera por prioridad y reasignación automática de cupos, justificando las estructuras de datos usadas y evaluando su desempeño.

II-B. Objetivos específicos

1. Ordenar a los estudiantes según los grupos de prelación con un único método de comparación, usando el IPA como dato de entrada.
2. Implementar una lista de espera por grupo que entregue siempre al estudiante de mayor prioridad: con una lista enlazada ordenada en la Entrega 1 y con un montículo en la Entrega 2.
3. Reasignar automáticamente cada cupo liberado al primero de la espera que haya aceptado la asignación y no tenga cruce de horario.
4. Sugerir grupos alternativos con cupo y sin cruce de horario.
5. Generar un reporte de los grupos con mayor demanda insatisfecha.
6. Medir, con 28.700 estudiantes simulados, el tiempo de las operaciones con cada implementación y comparar la asignación por prioridad con la asignación "gana quien recarga primero".

III. Alcance y limitaciones

III-A. Funcionalidades que se desarrollarán

Registro de grupos y estudiantes, clasificación por prioridad, procesamiento de la cita, carrito con deshacer, inscripción con control de cupo y de cruce de horario, lista de espera por prioridad, cancelación con reasignación automática, búsqueda por código, sugerencia de grupos alternativos, reporte de demanda y simulación con datos sintéticos.

III-B. Fuera del alcance

Interfaz gráfica, base de datos, conexión con el SIA real, posgrado, la asignación de las citas (el sistema usa su resultado), los sobrecupos, la cancelación extemporánea y el control de la bolsa de créditos.

III-C. Restricciones técnicas

Java 17, ejecución por consola y estructuras de datos propias, sin usar las colecciones de `java.util`.

III-D. Supuestos

- Solo se modela pregrado de la sede Bogotá, con datos simulados.
- Los datos de prioridad (discapacidad, vulnerabilidad, PAPA, avance e IPA) son datos de entrada. **El sistema no calcula el IPA**.
- La reasignación automática solo ocurre en las adiciones (fase 3); en la cancelación ordinaria (fase 4) nadie puede inscribir.
- Al pedir un grupo lleno, el estudiante indica si acepta que se le inscriba automáticamente cuando se libere un cupo. Solo quien acepta entra a la espera, y puede salir de ella cuando quiera.

III-E. Límites para el tiempo disponible

El proyecto avanza en tres entregas. La Entrega 1 implementa el flujo básico con estructuras lineales. Las entregas 2 y 3 reemplazan estructuras por versiones más eficientes y agregan horarios, sugerencias, reportes y experimentos.

IV. Requerimientos principales

La Tabla I lista los requerimientos funcionales y la entrega en que se implementa cada uno.

V. Trabajo relacionado inicial

Asignación por turnos. Atender a los estudiantes uno a uno en orden de prioridad corresponde al mecanismo de *dictadura serial*, estudiado por Abdulkadiroğlu y Sönmez [4]. Es el principio de las citas del SIA [2], pero solo se aplica en el momento de la cita.

Asignación de cursos. Budish y Cantillon [5] muestran que en la asignación de cursos por turnos de Harvard los estudiantes manipulan sus preferencias. Budish [6] propone un mecanismo de mercado que Wharton aplica como *Course Match* [7]. SISO-UNAL tiene un alcance más acotado: conserva la prioridad existente y la extiende a los cupos liberados.

Tabla I
Requerimientos funcionales

ID	Requerimiento	Entrega
RF1	Registrar grupos con su asignatura, número y cupo.	1
RF2	Registrar estudiantes con sus datos de prioridad y ubicarlos en su grupo de prelación.	1
RF3	Atender las solicitudes de una cita en orden (FIFO).	1
RF4	Agregar grupos al carrito y deshacer el último.	1
RF5	Inscribir si hay cupo y rechazar solicitudes repetidas.	1
RF6	Si el grupo está lleno y el estudiante acepta, ubicarlo en la espera por prioridad; permitirle salir de ella.	1
RF7	Al cancelar, asignar el cupo al primero de la espera.	1
RF8	Mostrar inscritos, cupos libres y espera de un grupo.	1
RF9	Validar cruces de horario al inscribir y al reasignar.	2
RF10	Retirar al estudiante de las demás esperas de la misma asignatura cuando obtiene cupo.	2
RF11	Buscar estudiantes y asignaturas por código en tiempo sublineal.	2
RF12	Sugerir otro grupo con cupo y sin cruce.	2
RF13	Reportar los grupos con más estudiantes en espera.	2
RF14	Simular el periodo de adiciones con 28.700 estudiantes y comparar estrategias.	3

Listas de espera académicas. Muchos sistemas de gestión académica tienen listas de espera que inscriben al siguiente estudiante cuando se libera un cupo, normalmente por orden de llegada. SISO-UNAL usa en su lugar la prioridad institucional.

Estructuras de datos. Un montículo binario inserta y extrae el máximo en $O(\log n)$; un árbol AVL busca en $O(\log n)$ y una tabla hash en $O(1)$ esperado [8], [9]. Estas estructuras guían la evolución del sistema.

VI. Diseño de la solución

VI-A. Arquitectura

El sistema tiene dos capas (Fig. 1). El **modelo** está formado por `Student`, que guarda los datos de prioridad y define el orden con `compareTo`, y `CourseGroup`, que guarda el cupo, los inscritos y la lista de espera, con las operaciones `request`, `leaveWaitlist` y `cancel`. Las **estructuras de datos** son genéricas y propias. La lista de espera se declara con la interfaz `PriorityQueueADT`, así que en la Entrega 2 se cambia por un montículo modificando una sola línea. `Main` ejecuta un escenario de demostración.

VI-B. Criterio de prioridad

Toda la regla de prioridad está en `Student.compareTo`, de costo $O(1)$. Primero compara el grupo de prelación (el menor va primero). Dentro del grupo, el 1 se ordena por PAPA de mayor a menor, el 2 por PAPA de menor a mayor (más riesgo primero), el 3 por avance y el 4 por IPA, ambos

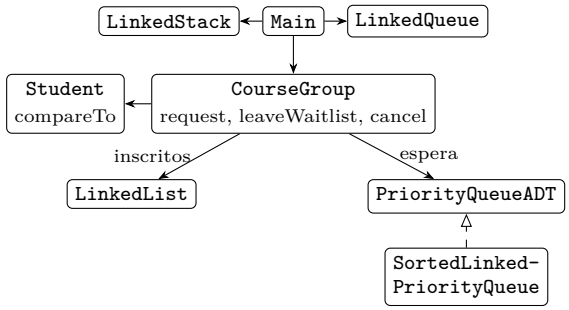


Figura 1. Clases de la Entrega 1. Las flechas sólidas indican uso; la punteada, implementación de la interfaz.

de mayor a menor. El último desempate es el código del estudiante, lo que da un orden total y reproducible.

VI-C. Flujo de información

1. La cola de prioridad ordena a los estudiantes para las citas, y en ese orden entran a la cola FIFO de la cita.
2. Cada solicitud se atiende con `request` (Algoritmo 1).
3. Al cancelar, `cancel` libera el cupo y lo asigna al primero de la espera (Algoritmo 2).
4. El carrito guarda grupos en una pila, y deshacer retira el último agregado.

Algoritmo 1 Solicitar un grupo (`request`)

Entrada: estudiante e, grupo g, aceptación a

if e ya está inscrito o en espera en g **then**
  rechazar
end if
if g tiene cupo **then**
  agregar e a $g.inscritos$ ▷ $O(1)$
  return Enrolled
else if a **then**
  insertar e en $g.espera$ ▷ $O(w)$
  return Waitlisted
else
  return Rejected
end if

Algoritmo 2 Cancelar y reasignar (`cancel`)

Entrada: estudiante e inscrito en el grupo g

eliminar e de $g.inscritos$ ▷ $O(c)$
if $g.espera$ no está vacía **then**
  $p \leftarrow g.espera$.removeHighest() ▷ $O(1)$
  agregar p a $g.inscritos$ ▷ $O(1)$
end if

VI-D. Ejemplo de funcionamiento

El grupo 1 de Cálculo Diferencial tiene cupo 2. En la cita, la cola de prioridad ordena a Ana (grupo 1), Carla (grupo 3), Elena (grupo 4, IPA 7,5) y Felipe (grupo 4, IPA 6,0).

Ana y Carla quedan inscritas. Elena acepta la asignación automática y entra a la espera; Felipe no la acepta y su solicitud se rechaza. En las adiciones llega Daniel (grupo 3), que acepta y queda *antes* que Elena por su prioridad, aunque llegó después. Cuando Carla cancela, el cupo pasa automáticamente a Daniel.

VII. Estructuras de datos seleccionadas

La Tabla II resume las estructuras de la Entrega 1; w es el tamaño de la lista de espera y c el cupo del grupo. Cada una responde a una necesidad concreta del flujo.

En conjunto, `request` cuesta $O(c + w)$, porque primero verifica que la solicitud no esté repetida; `cancel` cuesta $O(c)$ y `leaveWaitlist`, $O(w)$. En la Entrega 2, el montículo bajará la inserción en la espera a $O(\log w)$ y un árbol AVL permitirá buscar por código en $O(\log n)$. En la Entrega 3 se usarán una tabla hash y un grafo de prerrequisitos.

VIII. Estado de la implementación

Implementado (RF1 a RF8): las estructuras `Node`, `LinkedList`, `LinkedQueue`, `LinkedStack`, `PriorityQueueADT` y `SortedLinkedPriorityQueue`; la clasificación y el orden de prioridad en `Student`; y en `CourseGroup`, la inscripción con cupo, la lista de espera con aceptación previa, la salida de la espera y la cancelación con reasignación automática. El código compila con Java 17 y se probó en casos borde.

Parcialmente implementado: la demostración es guiada y no tiene menú; se trabaja con un grupo a la vez, sin catálogo; y `cancel` siempre reasigna, porque todavía no distingue la fase 3 de la fase 4.

Pendiente: cruces de horario, catálogo, retiro de las demás esperas, montículo, árbol AVL, sugerencia de grupos y reporte de demanda (Entrega 2); tabla hash, grafo de prerrequisitos y experimentos con 28.700 estudiantes (Entrega 3). Los resultados, la discusión y las conclusiones se desarrollarán en esas entregas.

Repositorio: https://github.com/byepesg/DS-262, carpeta `DS3/teams/team-07/projects/entrega-01/`. El `README.md` explica cómo compilar y ejecutar.

Declaración de uso de herramientas de IA

El equipo usó un asistente de inteligencia artificial (Claude, de Anthropic) para organizar la propuesta, revisar la redacción, revisar el código inicial e implementar la lista de espera por prioridad. Los integrantes verificaron el resultado y responden por él en la sustentación.

Referencias

[1] Dirección de Planeación y Estadística, Sede Bogotá, Universidad Nacional de Colombia, "Sede en cifras: Matriculados," 2026. [En línea]. Disponible en: https://planeacion.bogota.unal.edu.co/sede_en_cifras/reportes_interactivos/Matriculados/

[2] Vicerrectoría Académica, Universidad Nacional de Colombia, "Circular 13 de 2026: asignación de citas de inscripción," Bogotá, 2026.

[3] Dirección Nacional de Información Académica (DNINFOA), Universidad Nacional de Colombia, "Inscripción de asignaturas: infografía," 2026. [En línea]. Disponible en: https://dninfoa.unal.edu.co/docs/Infografia_inscripcion_asignaturas.pdf

[4] A. Abdulkadiroğlu y T. Sönmez, "Random serial dictatorship and the core from random endowments in house allocation problems," *Econometrica*, vol. 66, no. 3, pp. 689–701, 1998.

[5] E. Budish y E. Cantillon, "The multi-unit assignment problem: Theory and evidence from course allocation at Harvard," *American Economic Review*, vol. 102, no. 5, pp. 2237–2271, 2012.

[6] E. Budish, "The combinatorial assignment problem: Approximate competitive equilibrium from equal incomes," *Journal of Political Economy*, vol. 119, no. 6, pp. 1061–1103, 2011.

[7] E. Budish, G. P. Cachon, J. B. Kessler y A. Othman, "Course Match: A large-scale implementation of approximate competitive equilibrium from equal incomes for combinatorial allocation," *Operations Research*, vol. 65, no. 2, pp. 314–336, 2017.

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[9] M. T. Goodrich, R. Tamassia y M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: Wiley, 2014.

Tabla II
Estructuras de datos de la Entrega 1

Estructura	**Información**	**Operaciones**	**Por qué es apropiada**	**Alternativas**	**Costo**
Cola de prioridad sobre lista enlazada ordenada (`SortedLinkedPriorityQueue`)	Lista de espera de cada grupo; orden de las citas	insert, removeHighest, peekHighest, remove	El primero de la lista siempre es el de mayor prioridad, así que asignar un cupo es inmediato. Usa lo visto en el curso y en la Entrega 2 se reemplaza por un montículo.	Lista sin ordenar (insertar $O(1)$, extraer $O(w)$); montículo binario	insert $O(w)$; removeHighest $O(1)$; remove $O(w)$
Lista enlazada con referencia al último nodo (`LinkedList`)	Inscritos de un grupo	addLast, remove, contains	Crece sin límite fijo y agrega al final en tiempo constante.	Arreglo dinámico	addLast $O(1)$; remove y contains $O(c)$
Cola FIFO enlazada (`LinkedQueue`)	Estudiantes de una cita	enqueue, dequeue, peek	Atiende las solicitudes en el orden en que llegan.	Cola circular sobre arreglo	$O(1)$
Pila enlazada (`LinkedStack`)	Grupos del carrito	push, pop, peek	Deshacer retira siempre el último grupo agregado (LIFO).	Pila sobre arreglo	$O(1)$